

GitLab and SSH keys

Git is a distributed version control system, which means you can work locally but you can also share or "push" your changes to other servers. Before you can push your changes to a GitLab server you need a secure communication channel for sharing information.

The SSH protocol provides this security and allows you to authenticate to the GitLab remote server without supplying your username or password each time.

For a more detailed explanation of how the SSH protocol works, we advise you to read [this nice tutorial by DigitalOcean](#).

Locating an existing SSH key pair

Before generating a new SSH key pair check if your system already has one at the default location by opening a shell, or Command Prompt on Windows, and running the following command:

Windows Command Prompt:

```
type %userprofile%\\.ssh\id_rsa.pub
```

Git Bash on Windows / GNU/Linux / macOS / PowerShell:

```
cat ~/.ssh/id_rsa.pub
```

If you see a string starting with `ssh-rsa` you already have an SSH key pair and you can skip the generate portion of the next section and skip to the copy to clipboard step. If you don't see the string or would like to generate a SSH key pair with a custom name continue onto the next step.

Note that Public SSH key may also be named as follows:

- `id_dsa.pub`
- `id_ecdsa.pub`
- `id_ed25519.pub`

Generating a new SSH key pair

1. To generate a new SSH key pair, use the following command:

Git Bash on Windows / GNU/Linux / macOS:

```
ssh-keygen -o -t rsa -C "your.email@example.com" -b 4096
```

(Note: the `-o` option was introduced in 2014; if this command does not work for you, simply remove the `-o` option and try again)

Windows:

Alternatively on Windows you can download [PuttyGen](#) and follow [this documentation article](#) to generate a SSH key pair.

2. Next, you will be prompted to input a file path to save your SSH key pair to.

If you don't already have an SSH key pair use the suggested path by pressing enter. Using the suggested path will normally allow your SSH client to automatically use the SSH key pair with no additional configuration.

If you already have a SSH key pair with the suggested file path, you will need to input a new file path and declare what host this SSH key pair will be used for in your `~/.ssh/config` file, see [Working with non-default SSH key pair paths](#) for more information.

3. Once you have input a file path you will be prompted to input a password to secure your SSH key pair. It is a best practice to use a password for an SSH key pair, but it is not required and you can skip creating a password by pressing enter.

NOTE: **Note:** If you want to change the password of your SSH key pair, you can use `ssh-keygen -p -o -f <keyname>` . The `-o` option was added in 2014, so if this command does not work for you, simply remove the `-o` option and try again.

Adding a SSH key to your GitLab account

1. The next step is to copy the public SSH key as we will need it afterwards.

To copy your public SSH key to the clipboard, use the appropriate code below:

macOS:

```
pbcopy < ~/.ssh/id_rsa.pub
```

GNU/Linux (requires the xclip package):

```
xclip -sel clip < ~/.ssh/id_rsa.pub
```

Windows Command Line:

```
type %userprofile%\ssh\id_rsa.pub | clip
```

Git Bash on Windows / Windows PowerShell:

```
cat ~/.ssh/id_rsa.pub | clip
```

2. The final step is to add your public SSH key to GitLab.

Navigate to the 'SSH Keys' tab in your 'Profile Settings'. Paste your key in the 'Key' section and give it a relevant 'Title'. Use an identifiable title like 'Work Laptop - Windows 7' or 'Home MacBook Pro 15'.

If you manually copied your public SSH key make sure you copied the entire key starting with `ssh-rsa` and ending with your email.

3. Optionally you can test your setup by running `ssh -T git@example.com` (replacing `example.com` with your GitLab domain) and verifying that you receive a `Welcome to GitLab` message.

Working with non-default SSH key pair paths

If you used a non-default file path for your GitLab SSH key pair, you must configure your SSH client to find your GitLab private SSH key for connections to your GitLab server (perhaps `gitlab.com`).

For your current terminal session you can do so using the following commands (replacing `other_id_rsa` with your private SSH key):

Git Bash on Windows / GNU/Linux / macOS:

```
eval $(ssh-agent -s)
ssh-add ~/.ssh/other_id_rsa
```

To retain these settings you'll need to save them to a configuration file. For OpenSSH clients this is configured in the `~/.ssh/config` file for some operating systems. Below are two example host configurations using their own SSH key:

```
# GitLab.com server
Host gitlab.com
  RSAAuthentication yes
  IdentityFile ~/.ssh/config/private-key-filename-01

# Private GitLab server
Host gitlab.company.com
  RSAAuthentication yes
  IdentityFile ~/.ssh/config/private-key-filename
```

Due to the wide variety of SSH clients and their very large number of configuration options, further explanation of these topics is beyond the scope of this document.

Public SSH keys need to be unique, as they will bind to your account. Your SSH key is the only identifier you'll have when pushing code via SSH. That's why it needs to uniquely map to a single user.

Deploy keys

Per-repository deploy keys

Deploy keys allow read-only or read-write (if enabled) access to one or multiple projects with a single SSH key pair.

This is really useful for cloning repositories to your Continuous Integration (CI) server. By using deploy keys, you don't have to set up a dummy user account.

If you are a project maintainer or owner, you can add a deploy key in the project settings under the section 'Repository'. Specify a title for the new deploy key and paste a public SSH key. After this, the machine that uses the corresponding private SSH key has read-only or read-write (if enabled) access to the project.

You can't add the same deploy key twice using the form. If you want to add the same key to another project, please enable it in the list that says 'Deploy keys from projects available to you'. All the deploy keys of all the projects you have access to are available. This project access can happen through being a direct member of the project, or through a group.

Deploy keys can be shared between projects, you just need to add them to each project.

Global shared deploy keys

Global Shared Deploy keys allow read-only or read-write (if enabled) access to be configured on any repository in the entire GitLab installation.

This is really useful for integrating repositories to secured, shared Continuous Integration (CI) services or other shared services. GitLab administrators can set up the Global Shared Deploy key in GitLab and add the private key to any shared systems. Individual repositories opt into exposing their repository using these keys when a project maintainers (or higher) authorizes a Global Shared Deploy key to be used with their project.

Global Shared Keys can provide greater security compared to Per-Project Deploy Keys since an administrator of the target integrated system is the only one who needs to know and configure the private key.

GitLab administrators set up Global Deploy keys in the Admin area under the section **Deploy Keys**. Ensure keys have a meaningful title as that will be the primary way for project maintainers and owners to identify the correct Global Deploy key to add. For instance, if the key gives access to a SaaS CI instance, use the name of that service in the key name if that is all it is used for. When creating Global Shared Deploy keys, give some thought to the granularity of keys - they could be of very narrow usage such as just a specific service or of broader usage for something like "Anywhere you need to give read access to your repository".

Once a GitLab administrator adds the Global Deployment key, project maintainers and owners can add it in project's **Settings > Repository** section by expanding the **Deploy Key** section and clicking **Enable** next to the appropriate key listed under **Public deploy keys available to any project**.

NOTE: **Note:** The heading **Public deploy keys available to any project** only appears if there is at least one Global Deploy Key configured.

CAUTION: **Warning:** Defining Global Deploy Keys does not expose any given repository via the key until that repository adds the Global Deploy Key to their project. In this way the Global Deploy Keys enable access by other systems, but do not implicitly give any access just by setting them up.

Applications

Eclipse

How to add your SSH key to Eclipse: https://wiki.eclipse.org/EGit/User_Guide#Eclipse_SSH_Configuration

SSH on the GitLab server

GitLab integrates with the system-installed SSH daemon, designating a user (typically named `git`) through which all access requests are handled. Users connecting to the GitLab server over SSH are identified by their SSH key instead of their username.

SSH *client* operations performed on the GitLab server wil be executed as this user. Although it is possible to modify the SSH configuration for this user to, e.g., provide a private SSH key to authenticate these requests by, this practice is **not supported** and is strongly discouraged as it presents significant security risks.

The GitLab check process includes a check for this condition, and will direct you to this section if your server is configured like this, e.g.:

```
$ gitlab-rake gitlab:check
# ...
Git user has default SSH configuration? ... no
  Try fixing it:
  mkdir ~/gitlab-check-backup-1504540051
  sudo mv /var/lib/git/.ssh/id_rsa ~/gitlab-check-backup-1504540051
  sudo mv /var/lib/git/.ssh/id_rsa.pub ~/gitlab-check-backup-1504540051
  For more information see:
  doc/ssh/README.md in section "SSH on the GitLab server"
  Please fix the error above and rerun the checks.
```

Remove the custom configuration as soon as you're able to. These customizations are *explicitly not supported* and may stop working at any time.

Troubleshooting

If on Git clone you are prompted for a password like `git@gitlab.com's password:` something is wrong with your SSH setup.

- Ensure that you generated your SSH key pair correctly and added the public SSH key to your GitLab profile
- Try manually registering your private SSH key using `ssh-agent` as documented earlier in this document
- Try to debug the connection by running `ssh -Tv git@example.com` (replacing `example.com` with your GitLab domain)